

SIMATS ENGINEERING

ASSESSMENT TOOL 3

**COURSE NAME: Fundamentals of Data Science COURSE CODE:
DSA04**

COURSE OUTCOME COVERED

CO2: Apply data pre-processing and visualization techniques using Python libraries for effective data handling. (BL4)

QUESTIONS: 05 Total Marks:100

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I **Vivetha B 192424170** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Vivetha B

1. **An online food delivery company wants to analyse order data to improve delivery performance and customer satisfaction. Explain how:**
 - A. Pandas is used to read, clean, and filter order data
 - B. Data preprocessing is applied to handle missing delivery time, duplicate orders, and incorrect prices
 - C. Matplotlib or Seaborn is used to visualize delivery delays, top restaurants, and customer ratings

Conceptual explanation

An online food delivery company collects data about orders, restaurants, delivery times, prices, and customer ratings.

A. Use of Pandas

Pandas is used to read the order data and organize it in a table-like structure called a Data Frame. The company can use Pandas to:

- Read order data from files such as CSV.
- Check and clean the data.
- Filter delayed orders.
- Select orders from a particular restaurant.
- Analyse customer ratings and delivery times.

B. Data Preprocessing

Data preprocessing improves the quality of the order data.

- Missing delivery times can be filled using a suitable value such as the median delivery time.
- Duplicate orders can be removed so that the same order is not counted twice.
- Incorrect prices, such as negative or zero prices, can be identified and corrected or removed.

C. Visualization

Graphs help the company understand the data easily.

- A histogram or line chart can show delivery delays and delivery-time patterns.
- A bar chart can identify the restaurants receiving the highest number of orders.
- A bar chart or count plot can show customer ratings.

These insights help the company improve delivery speed, identify popular restaurants, and increase customer satisfaction.

File Edit Format Run Options Window Help

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
data = {
    "Order_ID": [101, 102, 103, 104, 105, 106, 107, 108],
    "Restaurant": [
        "Pizza Hub",
        "Burger King",
        "Pizza Hub",
        "Food Corner",
        "Burger King",
        "Pizza Hub",
        "Food Corner",
        "Pizza Hub"
    ],
    "Delivery_Time": [30, 55, None, 40, 60, 35, 45, 50],
    "Price": [250, 400, 300, -100, 500, 350, 450, 280],
    "Rating": [5, 4, 3, 5, 2, 4, 5, 3]
}
orders = pd.DataFrame(data)
print("ORIGINAL FOOD DELIVERY DATA")
print(orders)

orders["Delivery_Time"] = orders["Delivery_Time"].fillna(
    orders["Delivery_Time"].median()
)

orders = orders.drop_duplicates()

median_price = orders[orders["Price"] > 0]["Price"].median()

orders.loc[orders["Price"] <= 0, "Price"] = median_price

print("\nCLEANED FOOD DELIVERY DATA")
print(orders)

delayed_orders = orders[orders["Delivery_Time"] > 45]

print("\nDELAYED ORDERS")
print(delayed_orders)

plt.figure(figsize=(8, 5))

sns.histplot(
    orders["Delivery_Time"],
    bins=5,
    kde=True
```

```

plt.title("Distribution of Delivery Times")
plt.xlabel("Delivery Time (Minutes)")
plt.ylabel("Number of Orders")

plt.show()

top_restaurants = orders["Restaurant"].value_counts()

plt.figure(figsize=(8, 5))

top_restaurants.plot(kind="bar")

plt.title("Top Restaurants by Number of Orders")
plt.xlabel("Restaurant")
plt.ylabel("Number of Orders")

plt.xticks(rotation=45)

plt.show()

plt.figure(figsize=(8, 5))

sns.countplot(
    x="Rating",
    data=orders
)

plt.title("Customer Rating Distribution")
plt.xlabel("Customer Rating")
plt.ylabel("Number of Customers")

plt.show()

```

```

= RESTART: C:/Users/bvbvi/AppData/Local/Programs/Python/Python313/food_delivery_analysis.py
Matplotlib is building the font cache; this may take a moment.

```

ORIGINAL FOOD DELIVERY DATA

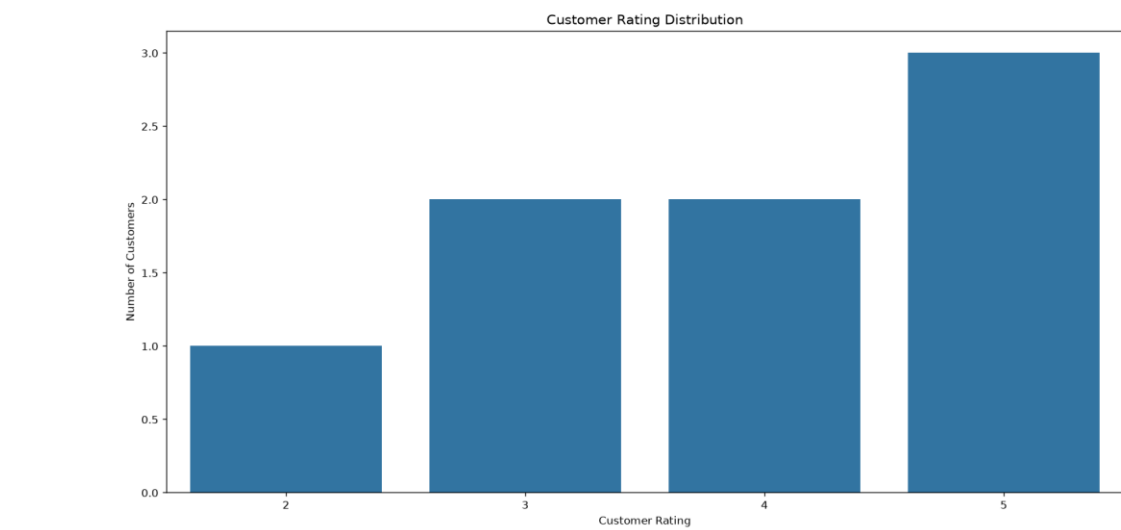
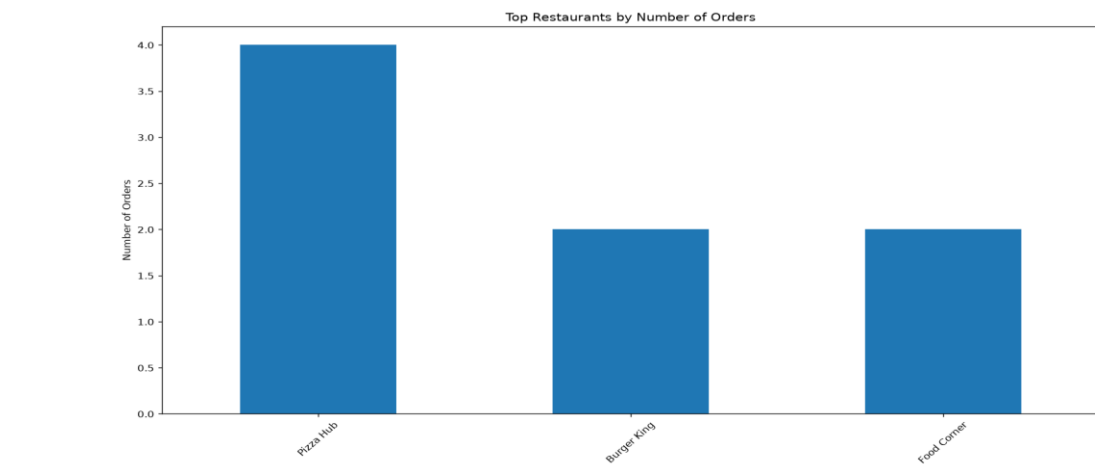
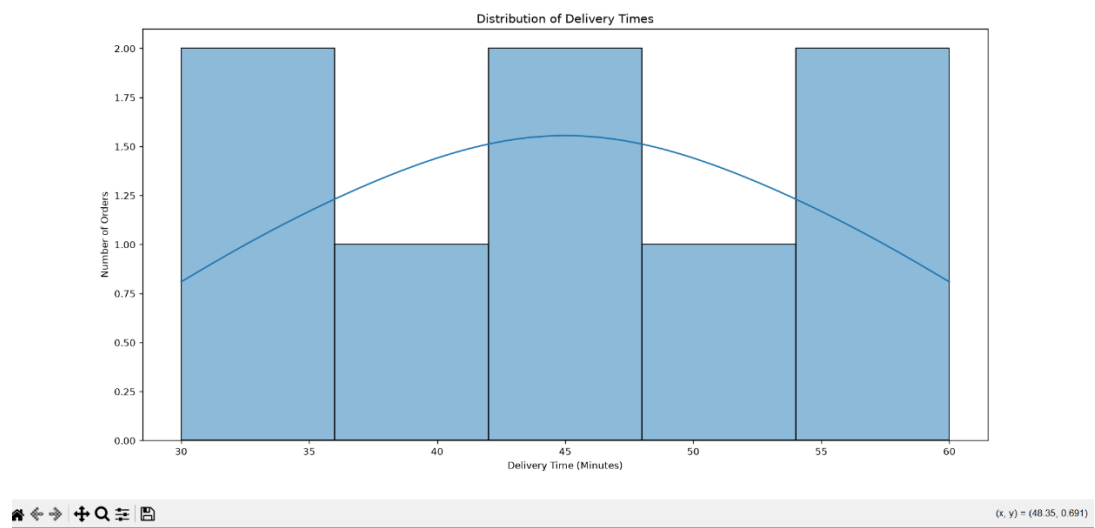
	Order_ID	Restaurant	Delivery_Time	Price	Rating
0	101	Pizza Hub	30.0	250	5
1	102	Burger King	55.0	400	4
2	103	Pizza Hub	NaN	300	3
3	104	Food Corner	40.0	-100	5
4	105	Burger King	60.0	500	2
5	106	Pizza Hub	35.0	350	4
6	107	Food Corner	45.0	450	5
7	108	Pizza Hub	50.0	280	3

CLEANED FOOD DELIVERY DATA

	Order_ID	Restaurant	Delivery_Time	Price	Rating
0	101	Pizza Hub	30.0	250	5
1	102	Burger King	55.0	400	4
2	103	Pizza Hub	45.0	300	3
3	104	Food Corner	40.0	350	5
4	105	Burger King	60.0	500	2
5	106	Pizza Hub	35.0	350	4
6	107	Food Corner	45.0	450	5
7	108	Pizza Hub	50.0	280	3

DELAYED ORDERS

	Order_ID	Restaurant	Delivery_Time	Price	Rating
1	102	Burger King	55.0	400	4
4	105	Burger King	60.0	500	2
7	108	Pizza Hub	50.0	280	3



2. Bank Loan Approval Analysis

A bank wants to study customer loan applications and identify patterns related to loan approval and rejection. Explain how:

- A. Pandas Data Frame operations are used to select income, loan amount, credit score, and approval status
- B. Preprocessing techniques are used to handle missing income values, encode gender/employment type, and normalize loan amount
- C. Visualization is used to compare approved and rejected applications based on income, credit score, and loan amount

Conceptual explanation

A bank collects information about customers who apply for loans, including income, loan amount, credit score, gender, employment type, and approval status.

A. Use of Pandas

Pandas Data Frames are used to select important columns such as:

- Customer income
- Loan amount
- Credit score
- Loan approval status

The bank can also filter approved and rejected applications and analyze patterns in the data.

B. Data Preprocessing

Preprocessing is required before analysing the loan data.

- Missing income values can be replaced with a suitable statistical value such as the median income.
- Gender and employment type are categorical values, so they can be converted into numerical values using encoding.
- Loan amounts can be normalized so that values are placed on a common scale.

This makes the data suitable for analysis and machine learning.

C. Visualization

Visualization helps compare approved and rejected loan applications.

- Income can be compared to find whether approved customers generally have higher income.
- Credit scores can be compared to identify their relationship with loan approval.
- Loan amounts can be compared to understand whether the requested amount affects approval.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

data = {
    "Customer_ID": [101, 102, 103, 104, 105, 106, 107, 108],
    "Income": [50000, 60000, None, 45000, 80000, 35000, 70000, 55000],
    "Loan_Amount": [200000, 250000, 150000, 180000, 300000, 120000, 280000, 220000],
    "Credit_Score": [750, 720, 680, 650, 800, 600, 780, 700],
    "Gender": ["Male", "Female", "Male", "Female",
               "Male", "Female", "Male", "Female"],
    "Employment_Type": ["Salaried", "Self-Employed", "Salaried",
                        "Unemployed", "Salaried", "Unemployed",
                        "Self-Employed", "Salaried"],
    "Approval_Status": ["Approved", "Approved", "Rejected", "Rejected",
                        "Approved", "Rejected", "Approved", "Approved"]
}

loans = pd.DataFrame(data)

print("ORIGINAL LOAN DATA")
print(loans)

selected_data = loans[
    [
        "Income",
        "Loan_Amount",
        "Credit_Score",
        "Approval_Status"
    ]
]

print("\nSELECTED LOAN INFORMATION")
print(selected_data)

loans["Income"] = loans["Income"].fillna(
    loans["Income"].median()
)

loans["Gender_Code"] = loans["Gender"].map({
    "Male": 0,
    "Female": 1
})

loans["Employment_Code"] = loans["Employment_Type"].map({
    "Unemployed": 0,
    "Salaried": 1,
    "Self-Employed": 2
})
```

```

loans["Loan_Amount_Normalized"] = (
    loans["Loan_Amount"] - loans["Loan_Amount"].min()
) / (
    loans["Loan_Amount"].max() - loans["Loan_Amount"].min()
)
print("\nPREPROCESSED LOAN DATA")
print(loans)
plt.figure(figsize=(8, 5))
sns.boxplot(
    x="Approval_Status",
    y="Income",
    data=loans
)
plt.title("Income Comparison: Approved vs Rejected Loans")
plt.xlabel("Loan Approval Status")
plt.ylabel("Income")
plt.show()
plt.figure(figsize=(8, 5))
sns.boxplot(
    x="Approval_Status",
    y="Credit_Score",
    data=loans
)
plt.title("Credit Score Comparison: Approved vs Rejected Loans")
plt.xlabel("Loan Approval Status")
plt.ylabel("Credit Score")
plt.show()
plt.figure(figsize=(8, 5))
sns.boxplot(
    x="Approval_Status",
    y="Loan_Amount",
    data=loans
)
plt.title("Loan Amount Comparison: Approved vs Rejected Loans")
plt.xlabel("Loan Approval Status")
plt.ylabel("Loan Amount")
plt.show()

```

===== RESTART: C:/Users/DVDVI/AppData/Local/Programs/Python

ORIGINAL LOAN DATA

	Customer_ID	Income	Loan_Amount	...	Gender	Employment_Type	Approval_Status
0	101	50000.0	200000	...	Male	Salaried	Approved
1	102	60000.0	250000	...	Female	Self-Employed	Approved
2	103	NaN	150000	...	Male	Salaried	Rejected
3	104	45000.0	180000	...	Female	Unemployed	Rejected
4	105	80000.0	300000	...	Male	Salaried	Approved
5	106	35000.0	120000	...	Female	Unemployed	Rejected
6	107	70000.0	280000	...	Male	Self-Employed	Approved
7	108	55000.0	220000	...	Female	Salaried	Approved

[8 rows x 7 columns]

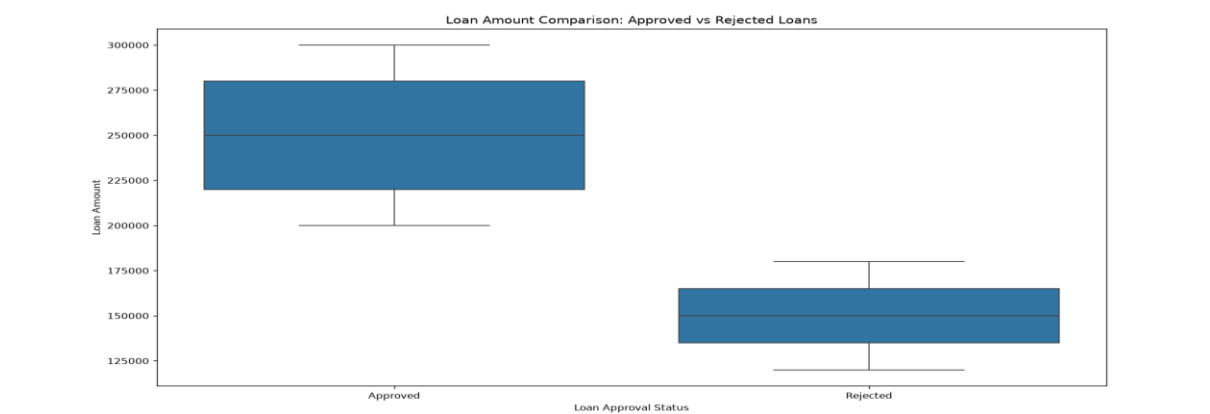
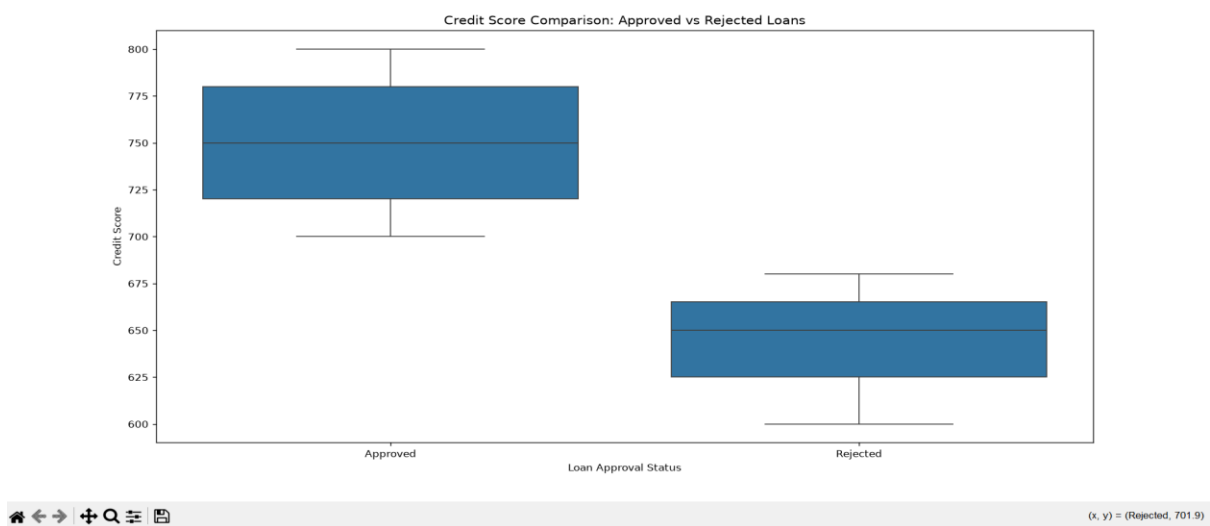
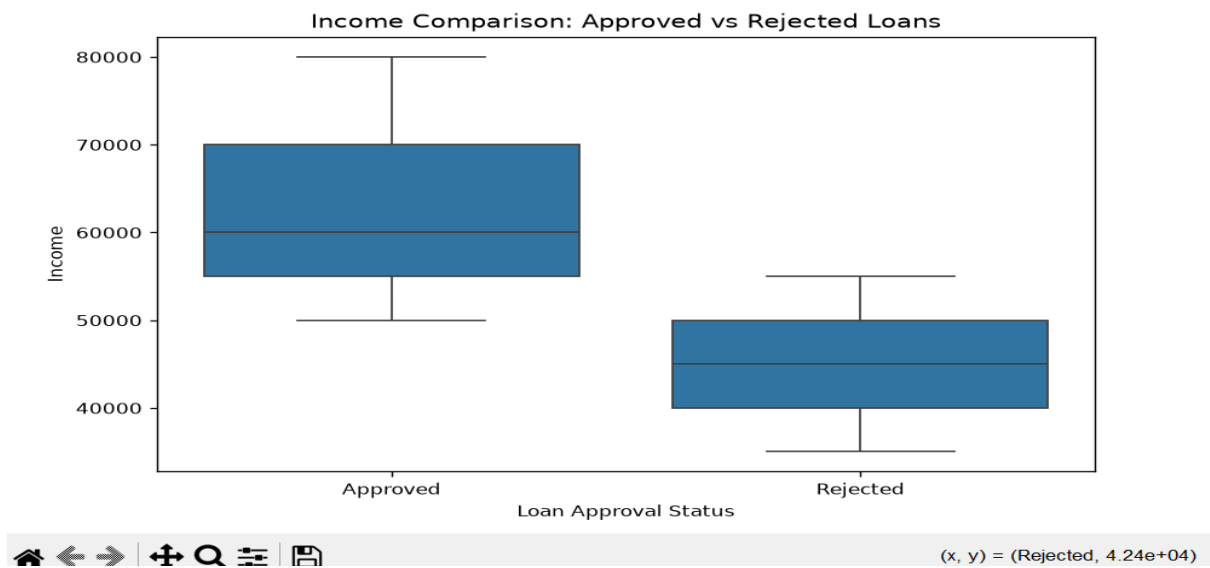
SELECTED LOAN INFORMATION

	Income	Loan_Amount	Credit_Score	Approval_Status
0	50000.0	200000	750	Approved
1	60000.0	250000	720	Approved
2	NaN	150000	680	Rejected
3	45000.0	180000	650	Rejected
4	80000.0	300000	800	Approved
5	35000.0	120000	600	Rejected
6	70000.0	280000	780	Approved
7	55000.0	220000	700	Approved

PREPROCESSED LOAN DATA

	Customer_ID	Income	...	Employment_Code	Loan_Amount_Normalized
0	101	50000.0	...	1	0.444444
1	102	60000.0	...	2	0.722222
2	103	55000.0	...	1	0.166667
3	104	45000.0	...	0	0.333333
4	105	80000.0	...	1	1.000000
5	106	35000.0	...	0	0.000000
6	107	70000.0	...	2	0.888889
7	108	55000.0	...	1	0.555556

[8 rows x 10 columns]



3. Smart Energy Consumption Monitoring

A smart home system collects daily electricity consumption data from different appliances such as AC, fan, light, and refrigerator. Explain how:

- A. Pandas is used to group electricity usage by appliance and date
- B. Data preprocessing is used to remove abnormal readings, fill missing values, and calculate total power consumption
- C. Line charts and bar charts are used to identify high-energy appliances and daily usage trends

Conceptual explanation

A smart home collects electricity usage data from appliances such as air conditioners, fans, lights, and refrigerators.

A. Use of Pandas

Pandas is used to group the energy data by:

- **Appliance** to find which appliance consumes the most electricity.
- **Date** to calculate daily electricity consumption.

Grouping the data helps identify energy usage patterns.

B. Data Preprocessing

The collected sensor data may contain errors.

- **Abnormal readings** can be identified and removed.
- **Missing energy readings** can be filled using a suitable value such as the median.
- The cleaned energy values can be added together to calculate **total power consumption**.

This ensures that the analysis is based on reliable data.

C. Visualization

Different charts help understand energy usage.

- A **bar chart** compares energy consumption among appliances and identifies high-energy appliances.
- A **line chart** shows daily electricity consumption over time.

This helps homeowners identify energy wastage and reduce electricity consumption.

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
data = {
    "Date": [
        "2026-07-01", "2026-07-01", "2026-07-01", "2026-07-01",
        "2026-07-02", "2026-07-02", "2026-07-02", "2026-07-02",
        "2026-07-03", "2026-07-03", "2026-07-03", "2026-07-03",
        "2026-07-04", "2026-07-04", "2026-07-04", "2026-07-04"
    ],
    "Appliance": [
        "AC", "Fan", "Light", "Refrigerator",
        "AC", "Fan", "Light", "Refrigerator",
        "AC", "Fan", "Light", "Refrigerator",
        "AC", "Fan", "Light", "Refrigerator"
    ],
    "Power_Consumption": [
        8.5, 2.0, 1.2, 3.5,
        9.0, 2.2, None, 3.6,
        8.8, 2.1, 1.3, 3.4,
        100, 2.3, 1.1, 3.7
    ]
}

energy = pd.DataFrame(data)

print("ORIGINAL ENERGY DATA")
print(energy)

appliance_usage = energy.groupby(
    "Appliance"
)["Power_Consumption"].sum()

print("\nTOTAL USAGE BY APPLIANCE")
print(appliance_usage)

daily_usage = energy.groupby(
    "Date"
)["Power_Consumption"].sum()

print("\nTOTAL DAILY USAGE")
print(daily_usage)

energy = energy[
    energy["Power_Consumption"] <= 50

```

```

energy["Power_Consumption"] = energy[
    "Power_Consumption"
].fillna(
    energy["Power_Consumption"].median()
)
total_consumption = energy[
    "Power_Consumption"
].sum()
print("\nCLEANED ENERGY DATA")
print(energy)
print("\nTOTAL POWER CONSUMPTION:")
print(total_consumption, "kWh")
appliance_usage = energy.groupby(
    "Appliance"
)["Power_Consumption"].sum()
plt.figure(figsize=(8, 5))
appliance_usage.plot(kind="bar")
plt.title("Energy Consumption by Appliance")
plt.xlabel("Appliance")
plt.ylabel("Power Consumption (kWh)")
plt.xticks(rotation=0)
plt.show()
daily_usage = energy.groupby(
    "Date"
)["Power_Consumption"].sum()

plt.figure(figsize=(8, 5))
daily_usage.plot(kind="line", marker="o")
plt.title("Daily Electricity Consumption Trend")
plt.xlabel("Date")
plt.ylabel("Power Consumption (kWh)")

plt.xticks(rotation=45)

plt.show()

```

```

= RESTART: C:/Users/bvbvi/AppData/Local/Programs/Python/Python313/energy_consumption_analysis.py
ORIGINAL ENERGY DATA

```

	Date	Appliance	Power_Consumption
0	2026-07-01	AC	8.5
1	2026-07-01	Fan	2.0
2	2026-07-01	Light	1.2
3	2026-07-01	Refrigerator	3.5
4	2026-07-02	AC	9.0
5	2026-07-02	Fan	2.2
6	2026-07-02	Light	NaN
7	2026-07-02	Refrigerator	3.6
8	2026-07-03	AC	8.8
9	2026-07-03	Fan	2.1
10	2026-07-03	Light	1.3
11	2026-07-03	Refrigerator	3.4
12	2026-07-04	AC	100.0
13	2026-07-04	Fan	2.3
14	2026-07-04	Light	1.1
15	2026-07-04	Refrigerator	3.7

```

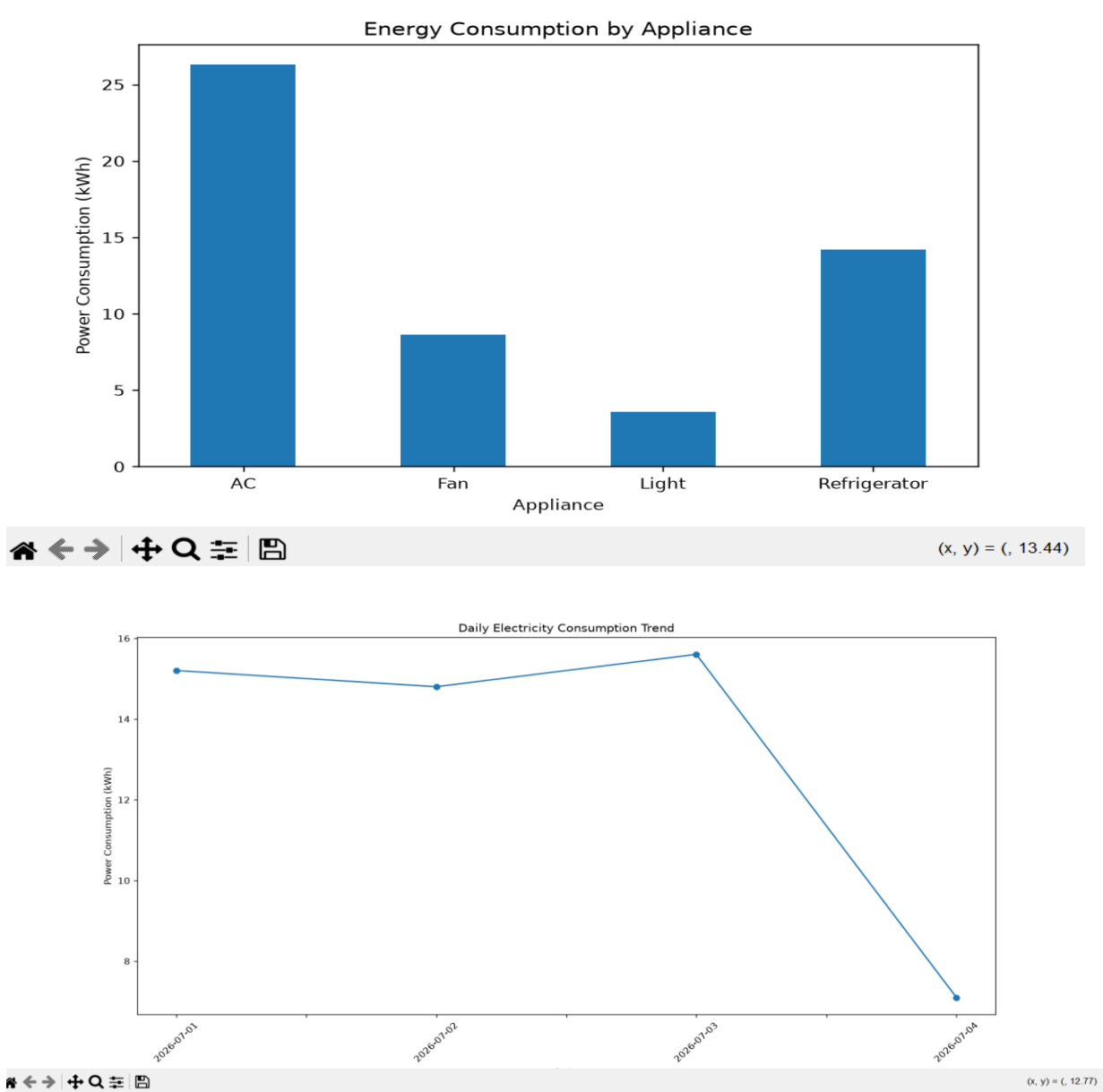
TOTAL USAGE BY APPLIANCE
Appliance
AC          126.3
Fan          8.6
Light        3.6
Refrigerator 14.2
Name: Power_Consumption, dtype: float64

TOTAL DAILY USAGE
Date
2026-07-01    15.2
2026-07-02    14.8
2026-07-03    15.6
2026-07-04   107.1
Name: Power_Consumption, dtype: float64

```

CLEANED ENERGY DATA			
	Date	Appliance	Power_Consumption
0	2026-07-01	AC	8.5
1	2026-07-01	Fan	2.0
2	2026-07-01	Light	1.2
3	2026-07-01	Refrigerator	3.5
4	2026-07-02	AC	9.0
5	2026-07-02	Fan	2.2
7	2026-07-02	Refrigerator	3.6
8	2026-07-03	AC	8.8
9	2026-07-03	Fan	2.1
10	2026-07-03	Light	1.3
11	2026-07-03	Refrigerator	3.4
13	2026-07-04	Fan	2.3
14	2026-07-04	Light	1.1
15	2026-07-04	Refrigerator	3.7

TOTAL POWER CONSUMPTION:
52.699999999999996 kWh



4. Online Learning Platform Performance Analysis

An online learning platform wants to analyse student activity such as login time, video completion, quiz score, and assignment submission. Explain how:

- A. Pandas is used to sort, filter, and rank students based on quiz scores and completion %
- B. Preprocessing is applied to handle missing quiz scores, convert date formats, and create new columns such as performance category
- C. Seaborn and Matplotlib are used to visualize student engagement, score distribution, and course completion status

Conceptual explanation

An online learning platform collects data about student activities such as login time, video completion, quiz scores, and assignment submissions.

A. Use of Pandas

Pandas is used to:

- **Sort** students according to their quiz scores.
- **Filter** students based on video completion percentage.
- **Rank** students according to quiz performance.

This helps the platform identify high-performing and low-performing students.

B. Data Preprocessing

The student data needs to be prepared before analysis.

- **Missing quiz scores** can be filled using a suitable statistical value.
- Login dates can be converted into a proper date format.
- A new **performance category** can be created, such as:
 - Excellent
 - Average
 - Needs Improvement

This makes it easier to analyse student performance.

C. Visualization

Visualization helps understand student engagement and performance.

- A **scatter plot** can show the relationship between login activity and video completion.
- A **histogram** can show the distribution of quiz scores.
- A **bar chart** can compare completed and incomplete courses.

These insights help the platform identify students who may need additional support.

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

data = {
    "Student_ID": ["S101", "S102", "S103", "S104",
                  "S105", "S106", "S107", "S108"],

    "Login_Hours": [10, 15, 8, 20, 12, 5, 18, 14],

    "Video_Completion": [90, 85, 60, 95, 75, 40, 88, 70],

    "Quiz_Score": [85, 78, None, 92, 65, 45, 88, 72],

    "Assignment_Submission": [
        "Yes", "Yes", "No", "Yes",
        "Yes", "No", "Yes", "Yes"
    ],

    "Course_Completion": [
        "Completed", "Completed", "Incomplete", "Completed",
        "Completed", "Incomplete", "Completed", "Incomplete"
    ],

    "Login_Date": [
        "2026-07-01", "2026-07-02", "2026-07-03",
        "2026-07-04", "2026-07-05", "2026-07-06",
        "2026-07-07", "2026-07-08"
    ]
}

students = pd.DataFrame(data)

print("ORIGINAL STUDENT DATA")
print(students)

sorted_students = students.sort_values(
    by="Quiz_Score",
    ascending=False
)

print("\nSTUDENTS SORTED BY QUIZ SCORE")
print(sorted_students)

high_completion = students[
    students["Video_Completion"] > 80
]

```

```
def performance_category(score):
    if score >= 80:
        return "Excellent"
    elif score >= 60:
        return "Average"
    else:
        return "Needs Improvement"
students["Performance_Category"] = students[
    "Quiz_Score"
].apply(
    performance_category)
print("\nPREPROCESSED STUDENT DATA")
print(students)
plt.figure(figsize=(8, 5))
sns.scatterplot(
    x="Login_Hours",
    y="Video_Completion",
    data=students)
plt.title("Student Engagement Analysis")
plt.xlabel("Login Hours")
plt.ylabel("Video Completion (%)")
plt.show()
plt.figure(figsize=(8, 5))
sns.histplot(
    students["Quiz_Score"],
    bins=5,
    kde=True)
plt.title("Quiz Score Distribution")
plt.xlabel("Quiz Score")
plt.ylabel("Number of Students")
plt.show()
plt.figure(figsize=(8, 5))
students["Course_Completion"].value_counts().plot(
    kind="bar")
plt.title("Course Completion Status")
plt.xlabel("Completion Status")
plt.ylabel("Number of Students")
plt.show()
plt.figure(figsize=(8, 5))
sns.countplot(
    x="Performance_Category",
    data=students)
plt.title("Student Performance Categories")
plt.xlabel("Performance Category")
plt.ylabel("Number of Students")
plt.show()
```



```

ORIGINAL STUDENT DATA
  Student_ID Login_Hours ... Course_Completion Login_Date
0      S101         10 ...      Completed 2026-07-01
1      S102         15 ...      Completed 2026-07-02
2      S103          8 ...      Incomplete 2026-07-03
3      S104         20 ...      Completed 2026-07-04
4      S105         12 ...      Completed 2026-07-05
5      S106          5 ...      Incomplete 2026-07-06
6      S107         18 ...      Completed 2026-07-07
7      S108         14 ...      Incomplete 2026-07-08

[8 rows x 7 columns]

STUDENTS SORTED BY QUIZ SCORE
  Student_ID Login_Hours ... Course_Completion Login_Date
3      S104         20 ...      Completed 2026-07-04
6      S107         18 ...      Completed 2026-07-07
0      S101         10 ...      Completed 2026-07-01
1      S102         15 ...      Completed 2026-07-02
7      S108         14 ...      Incomplete 2026-07-08
4      S105         12 ...      Completed 2026-07-05
5      S106          5 ...      Incomplete 2026-07-06
2      S103          8 ...      Incomplete 2026-07-03

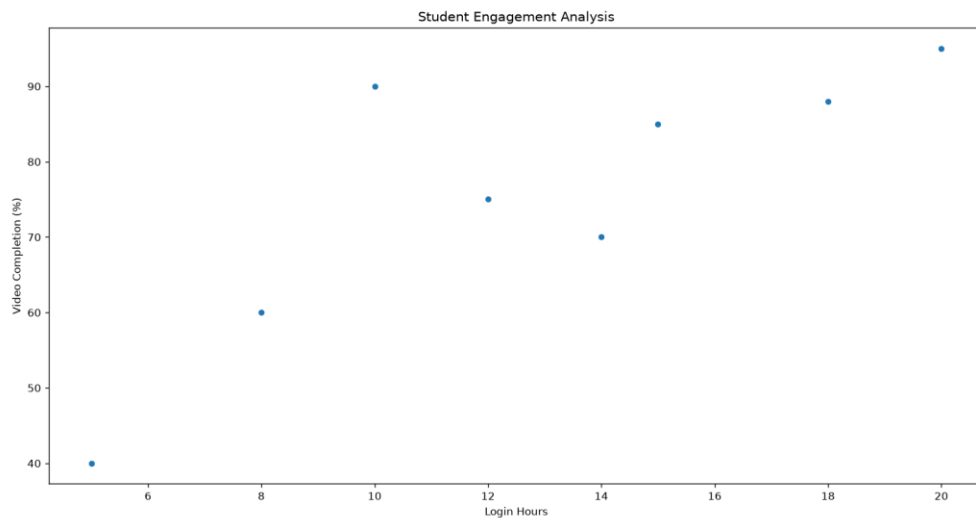
[8 rows x 7 columns]

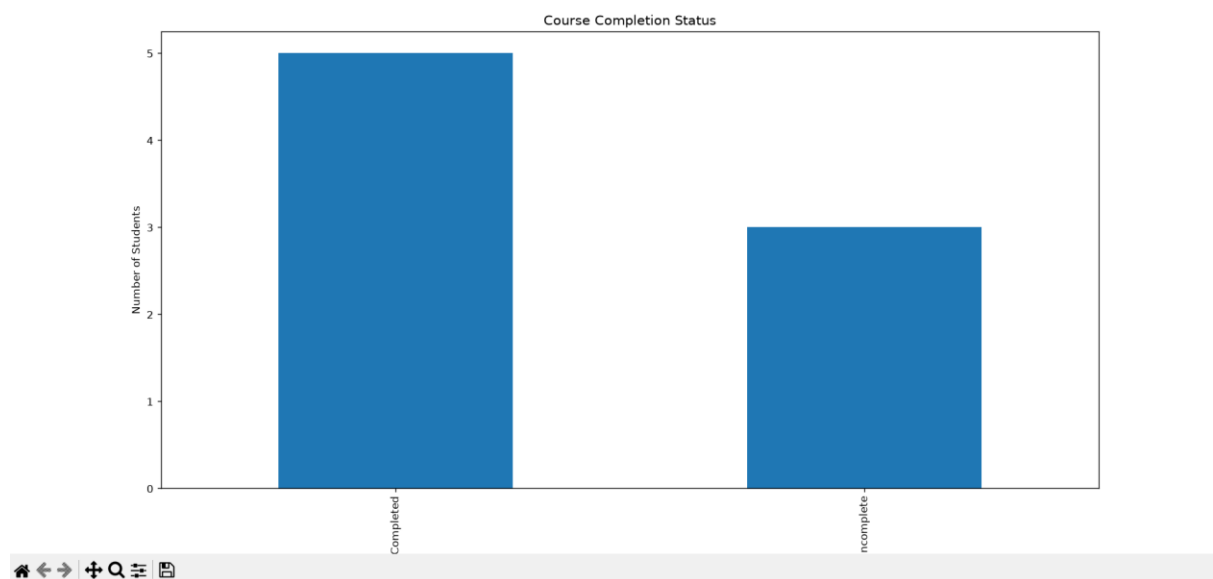
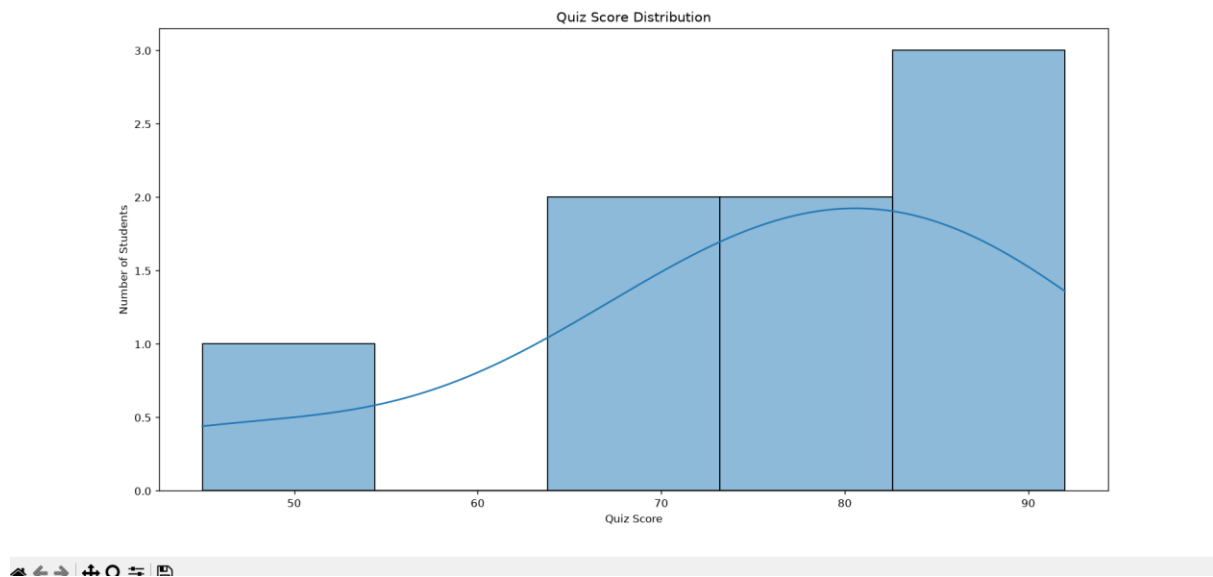
STUDENTS WITH HIGH VIDEO COMPLETION
  Student_ID Login_Hours ... Course_Completion Login_Date
0      S101         10 ...      Completed 2026-07-01
1      S102         15 ...      Completed 2026-07-02
3      S104         20 ...      Completed 2026-07-04
6      S107         18 ...      Completed 2026-07-07

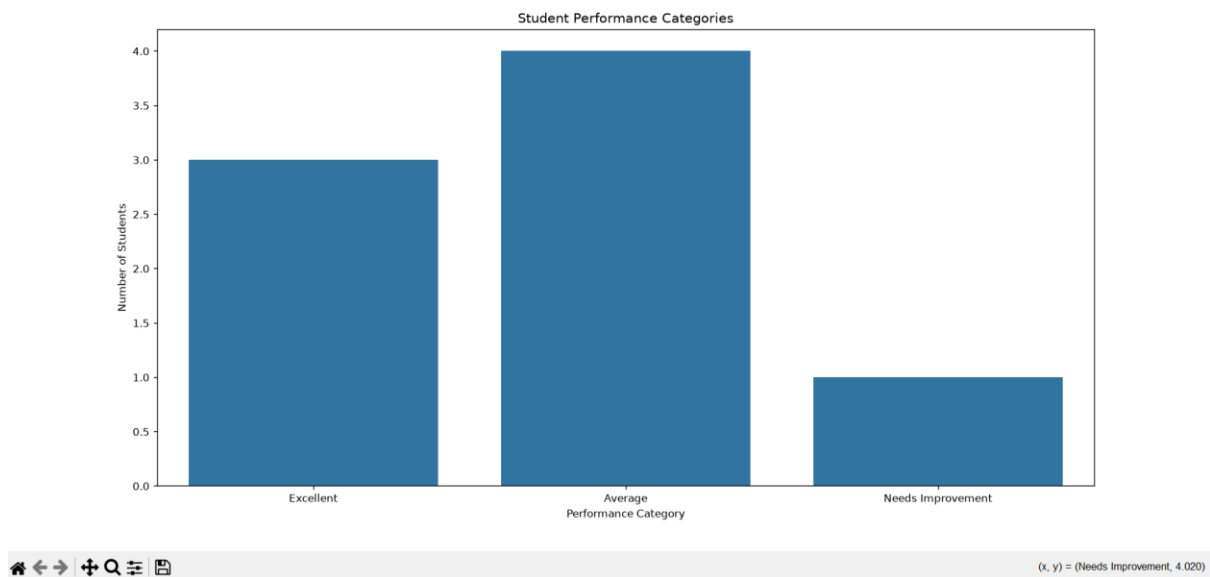
[4 rows x 7 columns]

STUDENT RANKING
  Student_ID Login_Hours ... Login_Date Quiz_Rank
0      S101         10 ... 2026-07-01      3.0
1      S102         15 ... 2026-07-02      4.0
2      S103          8 ... 2026-07-03      NaN
3      S104         20 ... 2026-07-04      1.0
4      S105         12 ... 2026-07-05      6.0
5      S106          5 ... 2026-07-06      7.0
6      S107         18 ... 2026-07-07      2.0
7      S108         14 ... 2026-07-08      5.0

```







5. Public Transport Passenger Data Analysis

A city transport department wants to analyse bus passenger data to improve route planning and reduce overcrowding. Explain how:

- A. Pandas is used to analyse passenger count by route, stop, and time slot
- B. Data preprocessing is used to clean invalid route numbers, handle missing passenger counts, and remove duplicate records
- C. Visualization techniques are used to show peak travel hours, busiest routes, and passenger flow patterns

Conceptual explanation

A city transport department collects data about bus routes, stops, time slots, and passenger counts.

A. Use of Pandas

Pandas is used to group and analyze passenger counts by:

- **Route** to find the busiest bus routes.
- **Stop** to identify stops with the highest passenger traffic.
- **Time slot** to identify peak travel hours.

This helps the transport department understand passenger movement.

B. Data Preprocessing

The passenger data must be cleaned before analysis.

- **Invalid route numbers** can be identified and removed.
- **Missing passenger counts** can be filled using a suitable statistical method.
- **Duplicate records** can be removed to prevent passengers from being counted more than once.

This improves the accuracy of the analysis.

C. Visualization

Charts help the transport department understand passenger flow.

- A **bar chart** can show peak travel hours.
- Another **bar chart** can identify the busiest routes.
- A **line chart or grouped bar chart** can show passenger flow patterns between different stops and time periods.

These insights help the city plan routes, increase buses during peak hours, and reduce overcrowding.

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
data = {
    "Date": [
        "2026-07-01", "2026-07-01", "2026-07-01",
        "2026-07-01", "2026-07-02", "2026-07-02",
        "2026-07-02", "2026-07-02", "2026-07-03",
        "2026-07-03", "2026-07-03", "2026-07-03"
    ],
    "Route": [
        101, 102, 103, 101,
        102, 103, 101, 102,
        103, 101, 102, 999
    ],
    "Stop": [
        "Central", "Market", "Station", "Central",
        "Market", "Station", "Airport", "Airport",
        "College", "College", "Station", "Unknown"
    ],
    "Time_Slot": [
        "Morning", "Morning", "Morning", "Evening",
        "Evening", "Evening", "Morning", "Morning",
        "Evening", "Night", "Night", "Night"
    ],
    "Passenger_Count": [
        80, 120, None, 150,
        130, 100, 200, 180,
        90, 60, 110, 50
    ]
}
transport = pd.DataFrame(data)
print("ORIGINAL BUS PASSENGER DATA")
print(transport)
transport = transport[
    transport["Route"].isin([101, 102, 103])
]
transport["Passenger_Count"] = transport[
    "Passenger_Count"
].fillna(
    transport["Passenger_Count"].median()
)
transport = transport.drop_duplicates()
print("\nCLEANED BUS PASSENGER DATA")
print(transport)

```

```

    transport["Passenger_Count"].median()
)
transport = transport.drop_duplicates()
print("\nCLEANED BUS PASSENGER DATA")
print(transport)
route_passengers = transport.groupby(
    "Route"
)[["Passenger_Count"]].sum()
print("\nPASSENGER COUNT BY ROUTE")
print(route_passengers)
stop_passengers = transport.groupby(
    "Stop"
)[["Passenger_Count"]].sum()
print("\nPASSENGER COUNT BY STOP")
print(stop_passengers)
time_passengers = transport.groupby(
    "Time_Slot"
)[["Passenger_Count"]].sum()
print("\nPASSENGER COUNT BY TIME SLOT")
print(time_passengers)
plt.figure(figsize=(8, 5))
time_passengers.plot(kind="bar")
plt.title("Passenger Count by Time Slot")
plt.xlabel("Time Slot")
plt.ylabel("Number of Passengers")
plt.xticks(rotation=0)
plt.show()
plt.figure(figsize=(8, 5))
route_passengers.plot(kind="bar")
plt.title("Passenger Count by Route")
plt.xlabel("Route Number")
plt.ylabel("Number of Passengers")
plt.xticks(rotation=0)
plt.show()
plt.figure(figsize=(8, 5))
sns.barplot(
    x="Time_Slot",
    y="Passenger_Count",
    hue="Stop",
    data=transport
)

plt.title("Passenger Flow by Stop and Time Slot")
plt.xlabel("Time Slot")
plt.ylabel("Passenger Count")
plt.show()

```

ORIGINAL BUS PASSENGER DATA

	Date	Route	Stop	Time_Slot	Passenger_Count
0	2026-07-01	101	Central	Morning	80.0
1	2026-07-01	102	Market	Morning	120.0
2	2026-07-01	103	Station	Morning	NaN
3	2026-07-01	101	Central	Evening	150.0
4	2026-07-02	102	Market	Evening	130.0
5	2026-07-02	103	Station	Evening	100.0
6	2026-07-02	101	Airport	Morning	200.0
7	2026-07-02	102	Airport	Morning	180.0
8	2026-07-03	103	College	Evening	90.0
9	2026-07-03	101	College	Night	60.0
10	2026-07-03	102	Station	Night	110.0
11	2026-07-03	999	Unknown	Night	50.0

CLEANED BUS PASSENGER DATA

	Date	Route	Stop	Time_Slot	Passenger_Count
0	2026-07-01	101	Central	Morning	80.0
1	2026-07-01	102	Market	Morning	120.0
2	2026-07-01	103	Station	Morning	115.0
3	2026-07-01	101	Central	Evening	150.0
4	2026-07-02	102	Market	Evening	130.0
5	2026-07-02	103	Station	Evening	100.0
6	2026-07-02	101	Airport	Morning	200.0
7	2026-07-02	102	Airport	Morning	180.0
8	2026-07-03	103	College	Evening	90.0
9	2026-07-03	101	College	Night	60.0
10	2026-07-03	102	Station	Night	110.0

PASSENGER COUNT BY ROUTE

Route

101 490.0

102 540.0

103 305.0

Name: Passenger_Count, dtype: float64

PASSENGER COUNT BY STOP

Stop

Airport 380.0

Central 230.0

College 150.0

Market 250.0

Station 325.0

Name: Passenger_Count, dtype: float64

PASSENGER COUNT BY STOP

Stop

Airport 380.0

Central 230.0

College 150.0

Market 250.0

Station 325.0

Name: Passenger_Count, dtype: float64

PASSENGER COUNT BY TIME SLOT

Time_Slot

Evening 470.0

Morning 695.0

Night 170.0

Name: Passenger_Count, dtype: float64

